

Documento de Especificação: Chat de Voz por Proximidade no DDNet

1. Introdução

Este documento detalha a implementação de um sistema de chat de voz por proximidade para o jogo DDraceNetwork (DDNet). O objetivo é fornecer um guia claro e passo a passo para uma Inteligência Artificial (IA) desenvolver essa funcionalidade, garantindo que o resultado final atenda aos requisitos de um chat de voz imersivo e funcional dentro do ambiente do jogo. O chat de voz por proximidade permitirá que os jogadores se comuniquem verbalmente com outros jogadores que estejam dentro de um determinado raio de distância no jogo, aumentando a imersão e a colaboração.

2. Visão Geral da Arquitetura do DDNet

O DDNet é um jogo de plataforma cooperativo de código aberto, construído sobre o motor do Teeworlds. Sua arquitetura é predominantemente escrita em C++ e utiliza a biblioteca SDL2 para manipulação de áudio, entrada e gráficos ¹.

2.1. Sistema de Rede

O jogo opera com um sistema de rede baseado em pacotes, onde as informações dos jogadores, como posição, movimento e ações, são constantemente sincronizadas entre o cliente e o servidor. A comunicação é otimizada para baixa latência, essencial para a jogabilidade em tempo real. A estrutura de rede existente precisará ser estendida para acomodar os dados de voz.

2.2. Sistema de Áudio

A SDL2 (Simple DirectMedia Layer) é a biblioteca principal para o gerenciamento de áudio no DDNet. Ela oferece funcionalidades de baixo nível para acesso ao hardware de áudio, permitindo a reprodução de sons e músicas, bem como a captura de áudio de dispositivos de entrada, como microfones ².

2.3. Gerenciamento de Posição dos Jogadores

O servidor do DDNet mantém o estado de todos os jogadores conectados, incluindo suas coordenadas espaciais. Essas informações são cruciais para determinar a proximidade entre os jogadores e, conseqüentemente, para a funcionalidade do chat de voz por proximidade.

3. Requisitos para Chat de Voz por Proximidade

Para implementar o chat de voz por proximidade, os seguintes requisitos funcionais e técnicos devem ser atendidos:

- **Captura de Áudio:** Capacidade de capturar áudio do microfone do jogador no cliente.
- **Codificação/Decodificação de Áudio:** Compressão eficiente do áudio capturado e descompressão do áudio recebido para minimizar o uso de largura de banda e manter a qualidade. O codec Opus é altamente recomendado devido à sua baixa latência e alta qualidade para voz ³.
- **Transmissão de Áudio via Rede:** Envio de dados de voz codificados do cliente para o servidor e do servidor para os clientes relevantes.
- **Decodificação e Reprodução de Áudio:** Reprodução do áudio decodificado no cliente, com controle de volume e posicionamento 3D.
- **Cálculo de Proximidade e Atenuação de Áudio:** Determinação dos jogadores dentro do raio de comunicação e ajuste do volume do áudio com base na distância e, idealmente, em fatores de oclusão.
- **Integração com o Sistema de Coordenadas do Jogo:** Utilização das posições dos jogadores no jogo para o cálculo de proximidade e áudio posicional.
- **Interface do Usuário (UI):** Feedback visual para o jogador sobre o status do microfone (ativo/inativo) e quem está falando.

4. Regras de Usabilidade e Privacidade

Para garantir uma experiência de usuário coesa e funcional, o chat de voz por proximidade deve aderir às seguintes regras:

4.1. Comunicação Baseada em Times

- **Escuta por Time:** Cada jogador só poderá ouvir os membros de seu próprio time. Isso significa que jogadores em times diferentes não se ouvirão, mesmo que estejam próximos.
- **Time Zero (Sem Time):** Jogadores que não pertencem a nenhum time (time zero) só poderão ouvir outros jogadores que também estão no time zero.

4.2. Áudio por Proximidade e Direcional

- **Distância de Audibilidade:** A voz será audível para jogadores a uma distância de aproximadamente uma tela. A audibilidade diminuirá progressivamente para duas ou três telas, tornando-se inaudível além desse limite.

- **Áudio Direcional:** O áudio será posicional (esquerda/direita/cima/baixo), fazendo com que a voz pareça vir da direção do jogador que está falando.
- **Posição da Voz:** A voz sempre emanará da posição do personagem do jogador, e não da direção para onde o jogador está olhando. Isso garante que a percepção espacial do áudio seja consistente com a localização física dos personagens no jogo.

4.3. Comportamento no Modo Espectador

- **Escuta do Espectador:** Ao observar um jogador, o espectador ouvirá os sons do local onde a câmera está focada (incluindo as vozes dos jogadores próximos à câmera) e também os sons próximos ao seu próprio personagem (se o espectador tiver uma posição virtual no mapa).
- **Invisibilidade de Voz do Espectador:** Espectadores não serão ouvidos pelos jogadores observados. Isso evita o que pode ser percebido como "vozes fantasmas" ou intrusões inesperadas na comunicação dos jogadores ativos.
- **Privacidade entre Espectadores:** Espectadores na mesma área não se ouvirão entre si, mantendo a privacidade e evitando confusão sonora entre eles.

4.4. Configurações do Lado do Cliente

Os jogadores devem ter acesso às seguintes configurações no menu do jogo:

- **Ajuste de Volumes:**
 - **Volume Geral do Chat de Voz:** Controle deslizante para ajustar o volume mestre de todas as vozes.
 - **Volume Perto/Longe:** Controles separados para ajustar o volume de vozes próximas e distantes, permitindo personalizar a atenuação.
 - **Distância de Escuta:** Um controle deslizante ou campo numérico para definir o raio máximo de audibilidade do chat de voz.
- **Mute Individual de Áudio:** Ao lado das opções de mute de chat de texto e emotes, deve haver uma opção para mutar individualmente o áudio de um jogador específico.
- **Controle de Volume por Pessoa:** Capacidade de ajustar o volume de cada jogador individualmente, além do mute.
- **Ouvir Apenas Amigos:** Uma opção para filtrar o chat de voz, permitindo que o jogador ouça apenas pessoas em sua lista de amigos (se houver um sistema de amigos implementado no DDNet).
- **Bloquear Áudio de Jogador Específico:** Uma funcionalidade para impedir que um jogador específico ouça o áudio do jogador local. O jogador local ainda poderá ouvir o

áudio do jogador bloqueado, mas o bloqueado não ouvirá o local. Isso oferece uma camada de privacidade e controle sobre quem pode ouvir a comunicação do jogador.

5. Plano de Implementação Detalhado

5.1. Módulo de Captura e Reprodução de Áudio (Cliente)

1. **Inicialização da SDL2 para Áudio:** Configurar a SDL2 para abrir um dispositivo de captura de áudio (microfone) e um dispositivo de reprodução de áudio. Definir parâmetros como taxa de amostragem (ex: 48kHz), formato de áudio (ex: `AUDIO_S16SYS`) e tamanho do buffer.
2. **Captura de Áudio:** Implementar um callback de áudio da SDL2 para ler continuamente os dados do microfone. Esses dados brutos serão passados para o módulo de codificação.
3. **Reprodução de Áudio:** Implementar um sistema para enfileirar e reproduzir os dados de áudio decodificados recebidos da rede. A SDL2 permite mixar múltiplas fontes de áudio, o que será essencial para ouvir vários jogadores simultaneamente.

5.2. Codificação e Decodificação de Áudio (Cliente e Servidor)

1. **Integração da Biblioteca Opus:** Adicionar a biblioteca Opus ao projeto DDNet. Isso envolverá a inclusão dos cabeçalhos e a vinculação das bibliotecas necessárias.
2. **Codificação de Áudio:** No cliente, os dados de áudio capturados pela SDL2 serão passados para o codificador Opus. O codificador converterá os dados de áudio brutos em pacotes de dados menores e mais eficientes para transmissão pela rede. Configurar o codificador para baixa latência e qualidade de voz ideal.
3. **Decodificação de Áudio:** No cliente, os pacotes de dados de voz recebidos da rede serão passados para o decodificador Opus. O decodificador converterá os pacotes de volta em dados de áudio brutos, que serão então enviados para o módulo de reprodução da SDL2.

5.3. Sistema de Rede para Voz (Cliente e Servidor)

1. **Definição de Novo Tipo de Pacote:** Criar uma nova estrutura de pacote de rede para encapsular os dados de voz codificados. Este pacote deve incluir:
 - Um identificador para o tipo de pacote de voz.
 - O ID do jogador remetente.
 - Os dados de áudio codificados (payload).
 - Um timestamp para ajudar na sincronização e mitigação de jitter.

2. **Transmissão do Cliente para o Servidor:** Quando um jogador fala (ex: pressiona uma tecla 'push-to-talk'), o cliente codifica o áudio e envia os pacotes de voz para o servidor.
3. **Processamento e Retransmissão do Servidor:** O servidor receberá os pacotes de voz. Para cada pacote, o servidor deve:
 - Identificar o jogador remetente, sua posição atual e seu time.
 - Iterar sobre todos os outros jogadores conectados, suas posições e seus times.
 - **Lógica de Times:** Verificar se o jogador remetente e o jogador receptor estão no mesmo time (ou ambos no time zero).
 - **Lógica de Bloqueio:** Verificar se o jogador receptor bloqueou o áudio do remetente ou se o remetente bloqueou o receptor de ouvi-lo.
 - Calcular a distância entre o remetente e cada jogador receptor potencial.
 - Se um jogador receptor estiver dentro do raio de proximidade configurado e as regras de time/bloqueio permitirem, o servidor retransmite o pacote de voz para esse jogador.
 - **Modo Espectador:** Se o receptor for um espectador, o servidor deve considerar a posição da câmera do espectador e a posição do próprio personagem do espectador para determinar a audibilidade, mas não retransmitir o áudio do espectador para jogadores ativos.
4. **Recebimento do Servidor para o Cliente:** Os clientes recebem os pacotes de voz retransmitidos pelo servidor, identificam o remetente e passam os dados de áudio para o decodificador Opus.

5.4. Cálculo de Proximidade e Áudio Posicional (Cliente)

1. **Obtenção de Posições dos Jogadores:** O cliente já recebe as posições de todos os jogadores do servidor. Essas informações serão utilizadas para o cálculo de proximidade.
2. **Cálculo de Distância:** Para cada pacote de voz recebido, o cliente calculará a distância entre a posição do jogador local e a posição do jogador remetente.
3. **Atenuação de Volume:** O volume do áudio do jogador remetente será atenuado com base na distância, utilizando as configurações de volume perto/longe definidas pelo usuário. Uma função de atenuação linear ou logarítmica pode ser usada. Por exemplo, o volume pode diminuir gradualmente até um ponto onde o áudio se torna inaudível, respeitando a 'distância de escuta' configurada. A estrutura `LinkedMem` do Mumble ⁴ pode servir como referência para como os dados posicionais (posição, vetor frontal, vetor superior) são expostos a aplicações externas, o que pode inspirar a forma como o DDNet gerencia esses dados internamente para o áudio posicional.

4. **Áudio 3D:** A SDL2 ou uma biblioteca de áudio adicional (como OpenAL ou Steam Audio SDK) será utilizada para áudio 3D. As posições dos jogadores serão usadas para espacializar o som, fazendo com que a voz pareça vir da direção correta no jogo (esquerda/direita/cima/baixo).

5.5. Interface do Usuário (UI) (Cliente)

1. **Indicador de Microfone Ativo:** Um pequeno ícone ou texto na UI que indica quando o microfone do jogador está ativo (ex: quando a tecla 'push-to-talk' é pressionada ou o modo de ativação por voz está ativo).
2. **Indicador de Fala:** Um ícone ou nome de usuário que aparece próximo ao jogador que está falando, visível para os jogadores dentro do raio de proximidade.
3. **Opções de Configuração:** Adicionar opções no menu do jogo para:
 - Ajustar o volume do chat de voz (geral, perto/longe).
 - Configurar uma tecla 'push-to-talk'.
 - Definir o raio de proximidade do chat de voz ('distância de escuta').
 - Ativar/desativar o chat de voz.
 - Opções de mute individual e controle de volume por pessoa.
 - Opção 'Ouvir Apenas Amigos'.
 - Opção 'Bloquear Áudio de Jogador Específico'.

6. Considerações Adicionais

- **Latência:** A baixa latência é crucial para uma experiência de chat de voz fluida. O codec Opus e um design de rede eficiente são fundamentais.
- **Consumo de Largura de Banda:** A codificação eficiente do áudio (Opus) e a retransmissão seletiva de pacotes pelo servidor (apenas para jogadores próximos e dentro das regras de time/bloqueio) ajudarão a minimizar o consumo de largura de banda.
- **Segurança:** Implementar medidas para prevenir spam de áudio ou abuso, como limites de taxa de pacotes por jogador ou opções de silenciar/bloquear jogadores.
- **Compatibilidade:** Garantir que a implementação seja compatível com as diversas plataformas suportadas pelo DDNet (Windows, Linux, macOS).

7. Conclusão

A implementação de chat de voz por proximidade no DDNet adicionará uma camada significativa de imersão e interação social ao jogo. Seguindo este plano detalhado, uma IA pode desenvolver essa funcionalidade, integrando captura/reprodução de áudio, codificação/decodificação eficiente, gerenciamento de rede e lógica de áudio posicional, além de respeitar as regras de usabilidade e privacidade. O resultado será uma experiência de comunicação rica e dinâmica para a comunidade DDNet.

8. Referências

[1] An intro to the DDraceNetwork game source code

[2] Simple DirectMedia Layer - Homepage

[3] Opus codec with android VOIP application | by (λx.x)eranga - Medium

[4] Integrate Mumble Positional Audio Through the Link Plugin | Mumble